
编译技术设计文档

21373090 张艺菲

词法分析

程序架构

1. word.java

```
private final String type; //标记该单词的符号

private final String value; //标记该单词的值

private final int line; //标记其行数，用于后续错误处理
```

2. sheet.java

```
public Sheet() {
    this.map = new HashMap<>();
    this.map.put("!", "NOT");
    this.map.put("*", "MULT");
    this.map.put "=", "ASSIGN";
    this.map.put("&&", "AND");
    this.map.put("/", "DIV");
    this.map.put(";", "SEMICN");
    this.map.put("||", "OR");
    this.map.put("%", "MOD");
    this.map.put(",", "COMMA");
    this.map.put("<", "LSS");
    this.map.put("(", "LPARENT");
    this.map.put("<=", "LEQ");
    this.map.put(")", "RPARENT");
    this.map.put(">", "GRE");
    this.map.put("[", "LBRACK");
    this.map.put(">=", "GEQ");
    this.map.put("]", "RBRACK");
    this.map.put("+", "PLUS");
    this.map.put("==", "EQL");
    this.map.put("{", "LBRACE");
    this.map.put("-", "MINU");
    this.map.put("!=", "NEQ");
}
```

```

        this.map.put("}", "RBRACE");
        this.map.put("Ident", "IDENFR");
        this.map.put("IntConst", "INTCON");
        this.map.put("DoubleConst", "DOUBLECON");
        this.map.put("FormatString", "STRCON");
        this.map.put("else", "ELSETK");
        this.map.put("void", "VOIDTK");
        this.map.put("main", "MAINTK");
        this.map.put("const", "CONSTTK");
        this.map.put("getint", "GETINTTK");
        this.map.put("int", "INTTK");
        this.map.put("printf", "PRINTFK");
        this.map.put("break", "BREAKTK");
        this.map.put("return", "RETURNTK");
        this.map.put("continue", "CONTINUETK");
        this.map.put("if", "IFTK");
        this.map.put("for", "FORTK");
        this.map.put("double", "DOUBLETK");
        //ADD 在这里补充了两个系统函数的名字
        this.map.put("do", "DOTK");
        this.map.put("while", "WHILETK");
    }

```

调用 `public String getCode(String tmp)` 函数，从该表中找到语法成分的类型，生成单词

3. `lexer`，主要生成的函数

`preProcess`：预处理，首先把注释和空行去掉

`lexicalAnalysis`：通过逐个字符分析，从而提取出不同的词法成分

`ArrayList<Word> words = new ArrayList<>()`：将提取出的单词存入其中

识别方法

- 单字符类

+	-	*	/	%	;	,
(	)	[	]	{	}	

- 双字符及其前缀类

!	>	<	=	&	
!=	>=	<=	==	&&	

在读取到首行的字符时，需要再尝试读取一次后面紧接着的一个字符，以确定它是单个出现还是和后面紧跟着的符号一起

- 格式字符串**FormatString**

以"开头，以"结尾的字符串。在读取到一个"时，直接向后查找到下一个"字符或文件结尾，两个"及它们之间的所有内容为一个格式字符串。

- 整常数**IntConst**

在开头读取到一个数字字符后，将所有连续的数字字符均算入。

- 标识符和英文保留字

在排除以上所有情况之后，若读取到一个a-z，或A-Z或-，则一路向后读取所有a-z，A-Z，0-9或-。

需要与定义的函数名称区分，在sheet.java里面已经实现

编码完成后的修改

编码完成后，发现自己对于/* */类注释的处理有误，进行了修改

语法分析

程序架构

生成了一棵以ASTNode为结点的树

1. ASTNode.java中，一共设置了以下属性，分别代表不同的语法成分：

```
COMPUNIT = 0, DECL = 1, CONSTDECL = 2, BTYPE = 3, CONSTDEF = 4,
          CONSTITVAL = 5, VARDECL = 6, VARDEF = 7, INITVAL = 8,
FUNCDEF = 9, FUNCTYPE = 10,
          FUNCFPARAMS = 11, FUNCFPARAM = 12, BLOCK = 13, BLOCKITEM =
14, STMT = 15,
          EXP = 16, COND = 17, LVAL = 18, PRIMARYEXP = 19, NUMBER =
20, UNARYEXP = 21,
          UNARYOP = 22, FUNCRPARAMS = 23, MULEXP = 24, ADDEXP = 25,
RELEXP = 26, EQEXP = 27,
          LANDEXP = 28, LOREXP = 29, CONSTEXP = 30, MAINFUNCDEF =
31, IDENT = 32,
          TOKEN = 32, BREAK = 33, CONTINUE = 34, RETURN = 35, PRINTF =
36,
          FORSTMT = 37, IF=38, FOR=39, GETINT=40, FORMATSTRING = 41;
```

在遍历的时候，每生成一个结点，都会将type改为相应的值，同时利用以下函数，将其存入子节点数组里，从而构建出一棵语法树

```

public void addSonNode(ASTNode tmp) {
    this.sonNodes.add(tmp);
}
private ArrayList<ASTNode> sonNodes;

```

同时,ASTNode 下方还有诸多属性, 当其作为某个特定语法成分的时候, 属性里面将会存相应的值

如该 ASTNode 是 NUMBER 类型, 那么

```

//如果是number, 那么它的value
private String number;

public void setNumber(String tmp) {
    this.number = tmp;
}

public int getNumber() {
    return Integer.parseInt(this.number);
}

```

就会在这里存值, 并通过这两个函数调用

其他属性列举如下:

```

//如果是IDENFR,那么它的value:
private String idValue;

//如果是funcType, 那么它的type
private String funcType;

//如果是number, 那么它的value
private String number;

//如果是unaryOp, 那么它的value
private String op;

//如果是funcFParam,那么它的dim
private int dim;

//如果是exp, 那么它的value以及是否被计算过了
private int constValue;
private boolean isCalculated;

//如果是formatSting
private Stringword stringword;

```

```
//TODO 返校以后新修改的
private int pos;
```

关键的属性 `pos`，代表该结点位于数据表的哪个位置，这是为了方便后续 `lval` 的取值设置的

这里之所以没有采用继承，而采用用 `type` 标记类型，是因为语法成分实在太多，继承写法很麻烦

2. `parser.java` 调用 `readCompUnit()` 函数，递归下降读取各语法成分

判断语法成分的方法和设计文档中的相同

`private ArrayList<Word> words`：词法分析中求出的单词表

`private int pos`：当前读到的单词位置

`private ASTNode globalCompUnit`：语法树根节点

编码完成后的修改

1. `constExp` 和 `exp` 的混用

2. 分析下一个语法成分的方法有误

主要表现在 `for` 的省略上，因为省略导致错把 “;” 读成了 `Ident`，导致出现了死循环

还比如在没有形参的时候，却仍然调用了读形参的函数，导致把 “(” 读成了 `Ident`，导致出现了死循环

3. `readUnaryExp` 中，对于下一种语法成分应该是什么的错误较多

错误处理

程序架构

1. `Error.java`

```
private String type; //a~m不同类型的错误

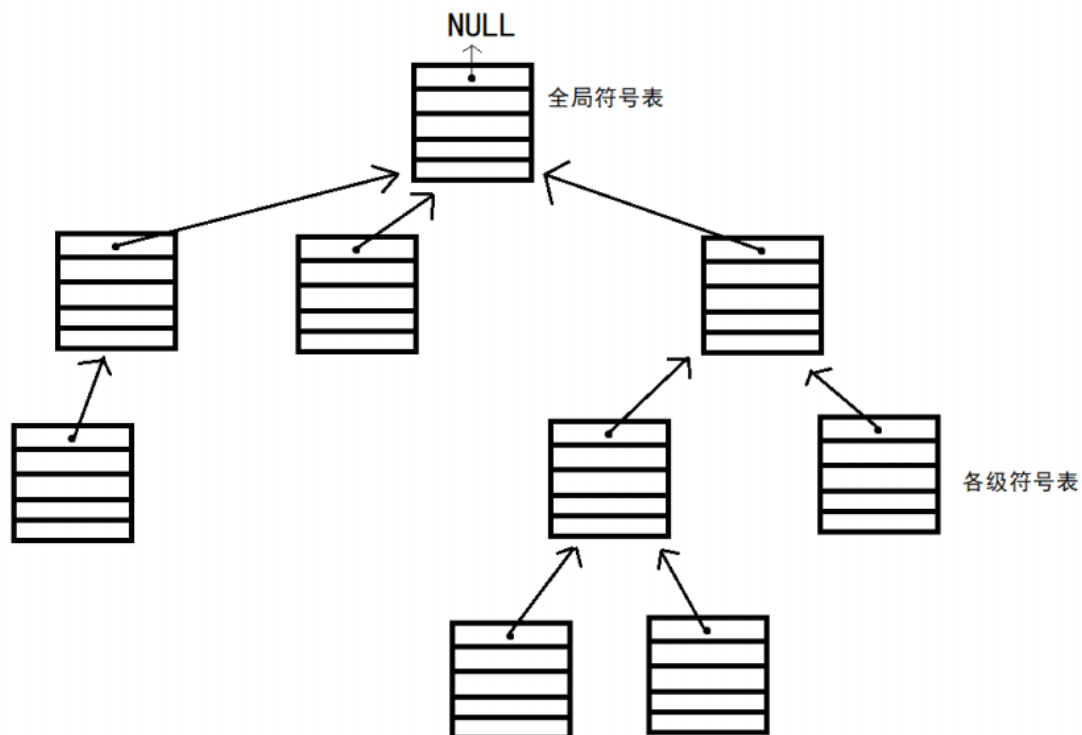
private int line; //错误的行号
```

2. `parser.java` 中新增属性和对应方法

```
private ArrayList<Error> errors;
public void printError() {
    collections.sort(errors);
    for (int i = 0; i < errors.size(); i++) {
        System.out.println(errors.get(i));
    }
}
```

检测到错误以后，在 `errors` 中 add 相应的错误

符号表的设计



在 `parser.java` 上修改，每个 `ASTNode` 绑定了一个符号表，这样就可以知道每个语法树结点对应的符号表是什么

1. `Symbol`：基本上作为一个抽象类使用，主要供 `Func` 和 `var` 继承
2. `SymbolFunc`

```
//false:void true:int
private boolean hasReturnValue;
//记录函数各个参数的维度
//其size代表参数数量
//除了id，包含了funcFParams的所有信息
private ArrayList<Integer> dims;
//return后面的值
public ASTNode returnValue;
//函数返回的标签
```

3. SymbolTable

```
private HashMap<String,SymbolVar> symbolVars;  
private HashMap<String,SymbolFunc> symbolFuncs;  
//为了lval专门设置的,只有变量,没有常数  
private ArrayList<SymbolVar> symbolVarList;  
//标记了变量和常数被引用  
private HashMap<SymbolVar,IRLabel> symbolVarRef;  
private HashMap<SymbolFunc,IRLabel> symbolFuncRef;  
//符号表和函数是多对一的关系  
private SymbolFunc nowFunction;  
//符号表的上一个  
private SymbolTable parent;  
private int parentPos;  
//错误处理用,用于记录当前是否位于循环体  
private boolean incycle;
```

4. SymbolVar

```
//是否是常数  
private boolean isConst;  
//维度,和每个维度对应的长度  
private int dim;  
private int[] dimLen;
```

编码完成后的修改

1. 发现自己在对形参赋值的时候报错了

```
int test(int a,int b) {  
    a = 1;  
}
```

这个情况不应该报错,但是我报错了,这源于自己对c语言语法知识的欠缺

2. 发现自己报的错误的顺序是乱的,后来为 Error 类增加了排序函数

中间代码生成

中间代码设计

参考部分往届设计

算术指令（允许立即数）

指令	用途
add <dest> <op1> <op2>	op1 + op2
minu <dest> <op1> <op2>	op1 - op2
mult <dest> <op1> <op2>	op1 * op2
div <dest> <op1> <op2>	op1 / op2（整除）
mod <dest> <op1> <op2>	op1 % op2（取模）
assign <dest> <op1>	dest = op1（赋值）
lshift <dest> <op1> <op2>	op1 << op2
rshift <dest> <op1> <op2>	op1 >> op2

逻辑指令

指令	用途
gre <dest> <op1> <op2>	op1 > op2
geq <dest> <op1> <op2>	op1 >= op2
lss <dest> <op1> <op2>	op1 < op2
leq <dest> <op1> <op2>	op1 <= op2
eql <dest> <op1> <op2>	op1 == op2
neq <dest> <op1> <op2>	op1 != op2

控制流

指令	用途
br <label>	无条件跳转至 <label>（op3）
bz <label> <op1>	若op1为0则跳转至<label>（op3）
bnz <label> <op1>	若op1不为0则跳转至<label>（op3）
call <ret> <func> <ParamList>	返回值（op3，可为null），函数调用（op1），带参数列表（可空）
setret <op3>	设置为返回值

指令	用途
ret	函数返回
exit	main函数结束，程序结束

内存操作

指令	用途
load <op3> <op1> <op2>	从op1为基地址，op2为偏移量处取值送op3
store <op3> <op1> <op2>	向op1为基地址，op2为偏移量处存op3
alloca <dest> <op1>	开辟op1字节大小空间，基地址送dest（int需*4）

输入输出

指令	用途
getint <dest>	dest=getint()
prints <strdest>	printf，带字符串地址（op3）
prints <op3>	printf，带整数（op3）

其他

指令	用途
#<id>	中间变量，编号为id
func <name> <paramlist>	函数定义（op3），带参数列表
label <LabelName>	标签（op3）

程序架构

`Calculator.java`：用于对常量进行计算

`IRSymbol.java`：抽象类，用于继承，以下四个类都继承于它

`IRArray`：生成的数组类

```
//数组基地址
private IRSymbol baseAddr;
//数组偏移地址
private IRSymbol offset;
```

`IRFunc`：生成的函数类

```
//函数的名字
private String name;
//入口
private IRSymbol entry;
//形参
private ArrayList<IRSymbol> fParams;
```

`IRImm`：生成的立即数类

```
private int value; //立即数的值
```

`IRLabel`：标签类

```
private int id; //互不相同
```

`IRLabelGenerator`：用于分配标签

`IRChar`：生成中间代码的指令

`toString` 函数可以利用以下属性生成对应的语句

```
private int type;

private IRSymbol op1;

private IRSymbol op2;

private IRSymbol op3;
```

实现流程

在 `parser` 部分，我们构建了以 `ASTNode` 为结点的树

在这部分，我们将会构建以 `IRSymbol` 为结点的树

1. 使用 `Calculator` 对代码中的常量进行计算
2. 使用 `IRGenerator` 将代码中的各部分生成中间式，下面是读到各部分的做法：
 - **变量**

遇到 `constDef` 中数组的情况，设置引用关系，存入哈希表 `constArray`

遇到 `varDef`，设置引用关系。如果是全局变量，存入哈希表 `globalVars`

其中，所有 `const` 类型的变量都在步骤1中求值过了。

针对非全局的 `varDef`，如果存在赋值过程，则计算 `initVal`，并存入相应的指令（普通数字是 `ASSIGN`，常数则是 `ALLOCA`）

◦ 函数

设置函数引用 `IRLabel`（代表函数的开头，将会解释函数的具体内容，同时跳转指令的位置也是这个标签）

设置函数出口 `IRLabel`（这个后面将会跟着 `RET`，代表函数的末尾）

读取函数形参并用 `IRSymbol` 表示，也有可能函数没有形参

把函数的名字作为 `key`，`IRFunc` 作为 `value` 存入 `hashmap` 中

生成函数描述指令（描述 `IRFunc`）

生成函数开头描述指令（描述函数引用 `IRLabel`）

生成函数内容指令（对 `block` 进行处理）

生成函数结尾描述指令（描述函数出口 `IRLabel`）

生成 `ret` 语句

◦ 函数形参

对每一个函数形参设置引用

◦ `Lval` 类语句

分两种情况，`lval=exp`，`lval=getint()`

`exp` 类

非数组：全局变量生成 `ASSIGN` 语句；非全局变量设置引用，生成 `ASSIGN` 语句

数组：统一生成 `STORE` 语句

`getint` 类

非数组：设置引用，生成 `getint` 语句

数组：生成临时变量，对临时变量的 `getint` 语句，临时变量对数组的 `store` 语句

◦ `if` 类语句

生成不满足 `if` 语句的跳转标签

生成不满足 `if` 语句的跳转语句（`genCond` 中实现）

生成满足 `if` 语句的相关语句（`genStmt` 中实现）

生成满足 `if` 语句，不执行 `else` 语句的跳转标签

生成满足 if 语句的跳转语句（无条件跳转）

生成不满足 if 语句的标签语句

生成不满足 if 语句的相关语句（genStmt 中实现）

生成满足 if 语句的跳转至末尾的语句

大致逻辑如下

```
if cond stmtA else stmtB stmtC
BZ cond #1
stmtA
BR #2
#1:
stmtB
#2:
stmtC
```

或者

```
if cond stmtA stmtC
BZ cond #1
stmtA
#1:
stmtC
```

◦ 循环类语句

这里介绍我的 for, break, continue 的做法,

for 语句

生成 for 循环开头标签

生成 for 循环结尾标签

生成 for 内语句开头标签

生成 ASSIGN 或者 STORE 类语句（如果有 stmt1）

生成 for 循环开头语句

生成 BZ 判断语句（如果有 cond）

生成 for 内语句开头语句

生成 for 内语句的语句

生成 ASSIGN 或 STORE 类语句（如果有 stmt2）

生成 BNZ 判断语句（如果有 cond）

大致逻辑如下：

```

for(forStmt1,cond,forStmt2) stmt
ASSIGN var1
#1:(stmt中存储)
BZ cond #2
#3:
stmt
ASSIGN var1
BNZ cond #3
#2:(stmt中存储)

```

也可以是:

```

for(forStmt1,,forStmt2) stmt
ASSIGN var1
#1:
#3:
stmt
ASSIGN var1
BR #3
#2:

```

break语句

每个 `stmt` 中都存储了 `startCycle` 和 `endCycle` , 如上所示
直接无条件跳转到 `endCycle`

continue 语句

重新执行一遍 `forStmt2` ,如果有的话
然后无条件跳转到 `startCycle`

◦ return语句

有 `exp` 生成 `SETRET` 语句, 然后再生成 `BR` 语句

◦ printf 语句

重要的地方是

```
printf("hi %d h %d ddd",exp1,exp2);
```

有两个部分, 分别要生成 `prints` 和 `printI` 语句

◦ cond 语句

要返回的是条件式的值的真伪性 (1代表 `true` , 0代表 `false`)

`1orExp` 大概逻辑如下:

```
lOrexp || lAndExp
```

用#1代表res

```
BNZ lOrexp #2
```

```
BNZ lAndExp #2
```

```
#1 = 0
```

```
BR #3
```

```
#2:
```

```
#1 = 1
```

```
#3:
```

其中被返回的是#1

#1在这个过程中已经被赋值过了

下文可以直接引用

`lAndExp` 大致逻辑如下:

```
lAndExp && EqExp
```

用#1代表res

```
BZ lAndExp #2
```

```
BZ EqExp #2
```

```
#1 = 1
```

```
BR #3
```

```
#2:
```

```
#1 = 0
```

```
#3:
```

`EqExp`, `RelExp` 返回的都是一些符号

- **Exp 语句**

总之就是层层递归, 没什么好说的

值得注意的是 `funcCall` 和 `lval` 部分

对于 `funcCall`, 能被调用的一定不是 `void` 类型的函数, 生成一条 `call` 语句即可, 同时要生成函数返回值标签

对于 `lval`, 要分为是否是数组两种情况来考虑

同时查找函数注意要找到当前位置最近的被赋值的情况

编码完成后的修改

1. `continue` 之后, `forStmt2` 的语句还需要再执行一遍, 之前我忽视了这一点, 导致了错误
2. 很多标签互相用错, 导致出现了死循环
3. 在进行 `print` 输出的时候, 要注意既有表达式又有字符串, 我一开始没有输出字符串, 只输出了表达式

4. 对于 `lval` 的查询有误，一开始我没有考虑到 `lval` 赋值的互相覆盖，导致出现了取错 `lval` 值的情况，之后在符号表中新建了数组，并在 `ASTNode` 中标记了位置，从而可以查询到最近一次的 `lval` 方法
5. 在初次写 `Calculator.java` 的时候，忽视了常量和非常量的区别，导致了不能求出的值也在求，如 `a+2` 等，最终出现了错误

MIPS代码生成

程序架构

在 `MIPSGenerator` 中

把以 `IRSymbol` 为结点的树生成最终语句，其属性如下：

```
//继承自IRGenerator
private ArrayList<IRChar> IRChars;
private HashMap<SymbolVar,IRSymbol> constMap;
private HashMap<SymbolVar,IRSymbol> globalMap;
private HashMap<IRSymbol,String> strMap;
//函数
private HashMap<String,Function> funcMap;
private Function nowFunc;
//全局标签
private HashSet<IRSymbol> labelSet;
//最后的结果
private StringBuilder mips;
```

在初始化的时候，需要对 `labelSet` 进行如下处理：

```
this.labelSet.addAll(strMap.keySet());
this.labelSet.addAll(constMap.values());
this.labelSet.addAll(globalMap.values());
```

为的是后面区分是否需要把 `IRChar` 中涉及的变量添加进 `Function`

新建了函数类，用于标记现在的函数，方便对栈帧进行操作

`Function.java`

```
//函数名称
private String name;
//函数形参
private ArrayList<IRSymbol> fParams;
//函数参数位于栈中的位置
private HashMap<IRSymbol,Integer> symbolOff;
```

编码完成后的修改

未做优化，用了四个寄存器来进行值的存储，比较简单

一开始的时候没有注意到地址的取整问题，导致经常会报RE的错误，随后参照往届代码进行了修改